DEEP LEARNING · SEQUENCE MODELS

The Transformer

Nine problems, one architecture: how researchers escaped the limits of the RNN, one problem at a time.

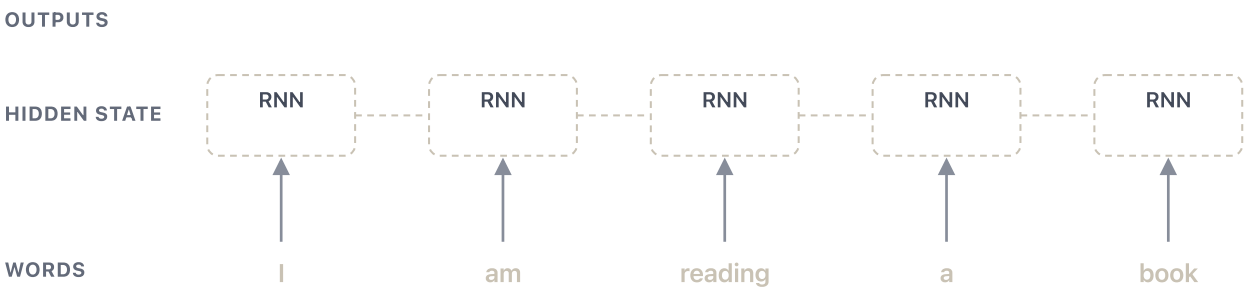
TASK I am reading a book → मैं एक किताब पढ़ रहा हूँ

---

0 Recap: what an RNN does

Last class we built the RNN. It reads a sentence in order, one word per step.

The RNN reads one word per step Reset Previous Next Play



The RNN reads “I am reading a book” from left to right. Press **Next**.

One word at a time
The RNN reads the sentence in order, one word per step.

A running summary
The hidden state carries everything read so far to the next step.

One output per step
Each step produces an output from the current word and the summary.

Fading memory
Gradients shrink across many steps, so early words are hard to learn from.

KNOWLEDGE CHECK

A plain RNN reads a 40-word sentence. Why does it struggle to use the first word when producing the last output?

A Gradients shrink as they flow back through many time steps

B Each hidden state is computed from the previous hidden state

C The same weight matrices are shared across every time step

D The network produces one output for every input word it reads

PROLOGUE · LONG BEACH · DECEMBER 2017

A poster session that would not end

At NeurIPS 2017, the evening poster session was so crowded that the eight authors of one paper talked until they were hoarse. At 10:30 pm, when it closed, a crowd was still there, and security had to ask them to leave.



Sepp Hochreiter
Co-inventor of the LSTM
In 1991, as a student, he diagnosed the vanishing gradient problem you studied last week. That evening he walked over to praise the paper that had just replaced his model.

TODAY **How do you escape a problem instead of patching it?**

PART 1 · 2014

The RNN breaks

1 An RNN cannot translate

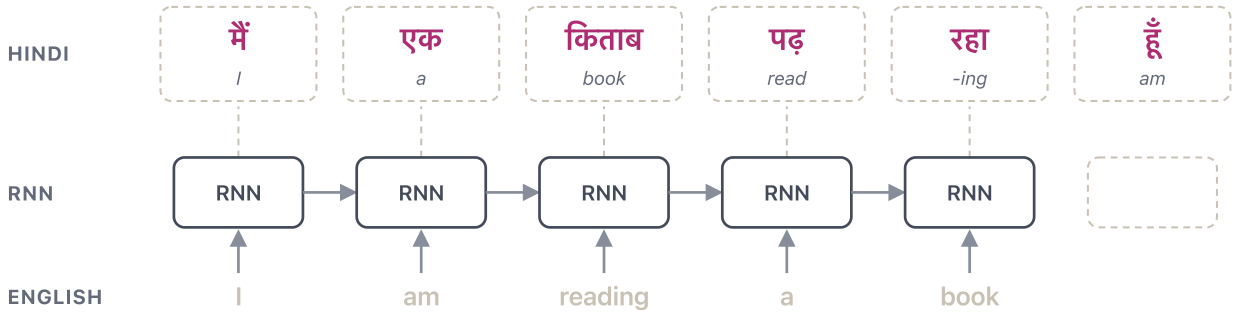
Problem 1 of 9

Can the RNN we built translate "I am reading a book" into Hindi?

PROBLEM The RNN writes one Hindi word at each step, in step with the English words. Hindi needs six words, in a different order.

One Hindi word per step Reset Previous Next Play

I am reading a book → मैं एक किताब पढ़ रहा हूँ



The RNN must write the Hindi sentence on top, one word at each step, in step with the English words below. Press **Next**.

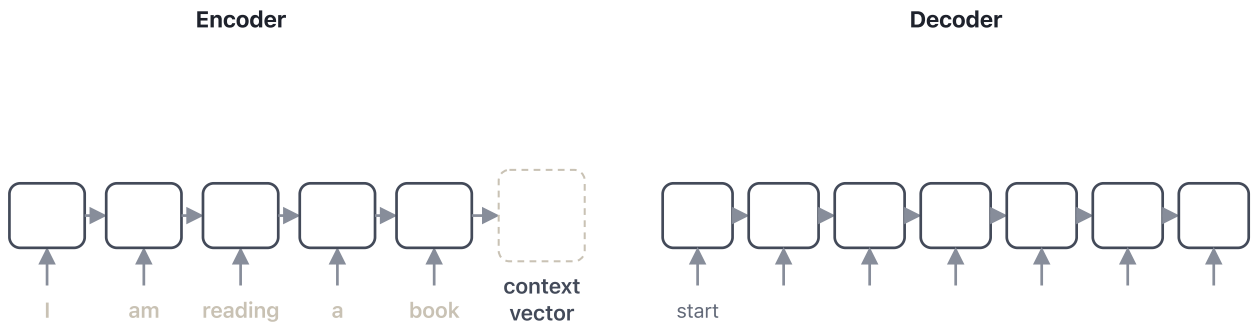
> **KEY IDEA** **Translation does not go word by word in order, so the output cannot be produced in step with the input.**

Fix · Read everything, then write

FIX Use two RNNs. The **encoder** reads the whole English sentence. The **decoder** then writes the Hindi sentence, one word at a time.

Read everything, then write Reset Previous Next Play

I am reading a book → मैं एक किताब पढ़ रहा हूँ



Two RNNs. The **encoder** reads the English sentence. The **decoder** writes the Hindi sentence.

KEY IDEA **Lengths no longer need to match, and nothing is needed from the future.**

STORY · 2014

A race between two labs



Ilya Sutskever
Google, Mountain View · 8 GPUs
His team was training a sequence-to-sequence LSTM for translation.



Yoshua Bengio
Université de Montréal · 1 GPU
His lab, with Kyunghyun Cho, ran its translation model on far less hardware, and knew Google was ahead.

2 One vector carries the whole sentence

Problem 2 of 9

The encoder-decoder works on short sentences. Now give it a long one.

PROBLEM The decoder sees the English sentence only through one context vector of fixed size.

Everything through one vector Reset Previous Next Play

one vector, fixed size



The decoder will see the English sentence only through this one vector. Its size is fixed.

KEY IDEA **One fixed-size vector, carried through many steps, cannot hold a long sentence.**

STORY · MONTREAL · 2014

The intern who doubted the vector



Dzmitry Bahdanau
Intern, Yoshua Bengio's lab
He had finished the first year of a master's degree in Germany, told Bengio he would work on anything, and was put on machine translation with Kyunghyun Cho.

His doubt. Squeezing a whole sentence into one vector seemed wrong to him from the start. He wanted a PhD offer, so he kept quiet and did what he was good at: writing code and fixing bugs.

Which English word does किताब need?

Think first. The decoder is about to write किताब (kitaab). Which English word does it need? Show the answer

First attempt: move in lockstep

Pair word i with word i Reset Previous Next Play

I am reading a book → मैं एक किताब पढ़ रहा हूँ

मैं	एक	किताब	पढ़	रहा	हूँ
I	*a*	*book*	*read*	*-ing*	*am*
I	am	reading	a	book	no word

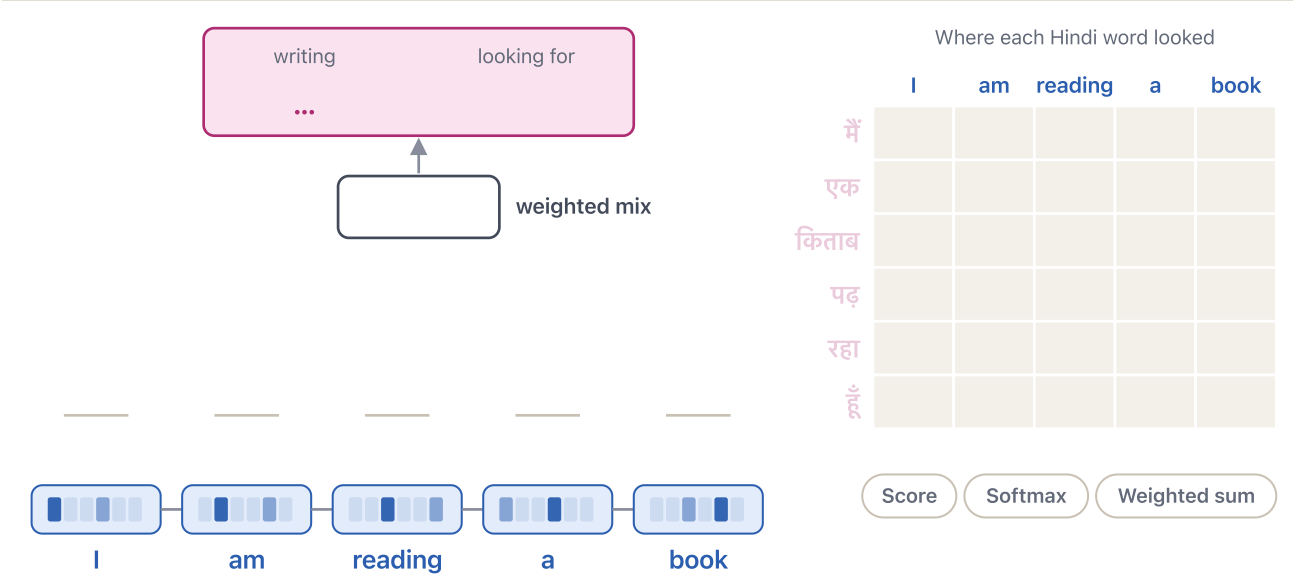
First attempt: pair Hindi word 1 with English word 1, word 2 with word 2, and so on.

Fix · Let the decoder look back

FIX Keep every encoder state. At each step, the decoder looks back at all of them and takes a weighted mix. This is **attention**.

The decoder looks back Reset Previous Next Play

I am reading a book → मैं एक किताब पढ़ रहा हूँ



The encoder now keeps its state for every English word, not only the last one. Each state is drawn as a pattern of six cells. Press **Next**.

1 **Score**
Compare the decoder's current state with every encoder state.

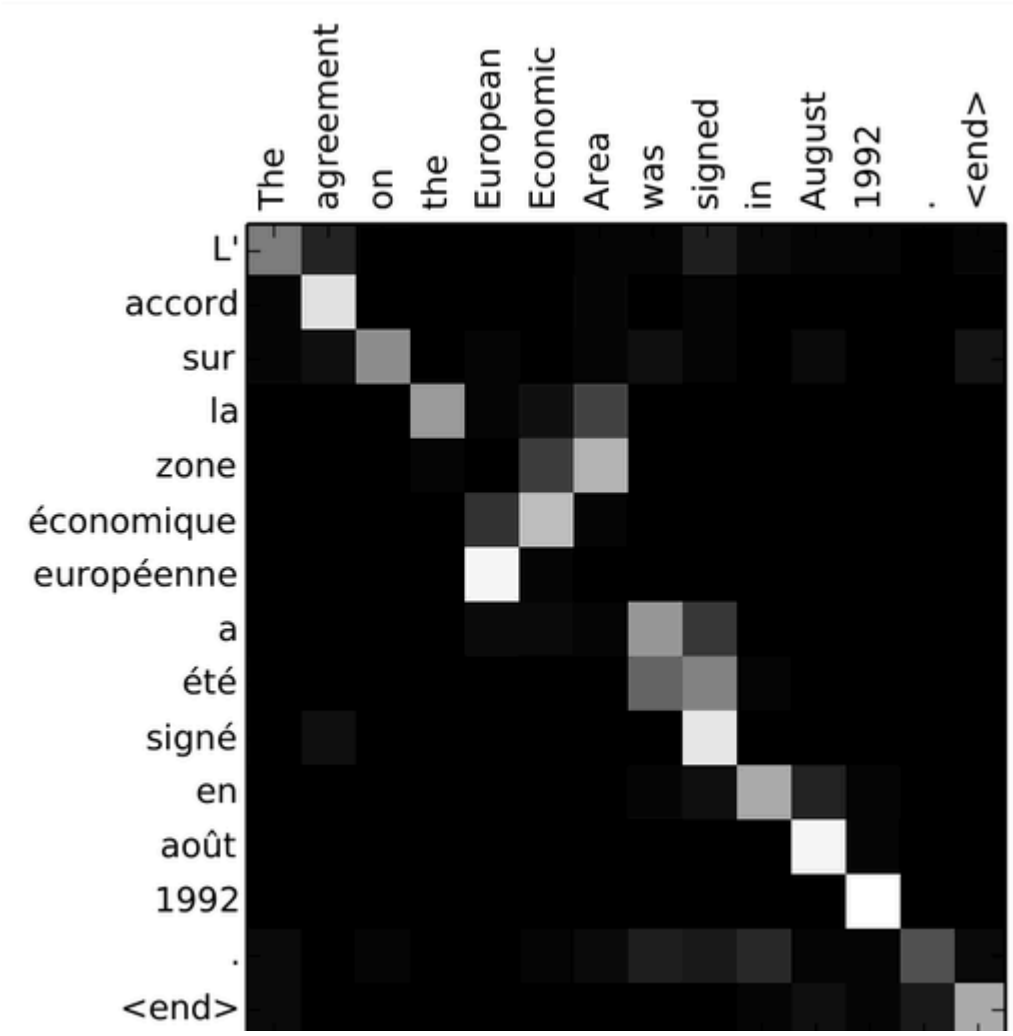
2 **Softmax**
Turn the scores into weights that add up to 1.

3 **Weighted sum**
Mix the encoder states with those weights, and hand the mix to the decoder.

> KEY IDEA **Every output word gets its own view of the input.**

The model learns where to look

Nobody tells the model which words match. It is trained only to translate, and it finds the alignment because looking at the right word helps. Here is a real alignment from Bahdanau's paper, from English to French.



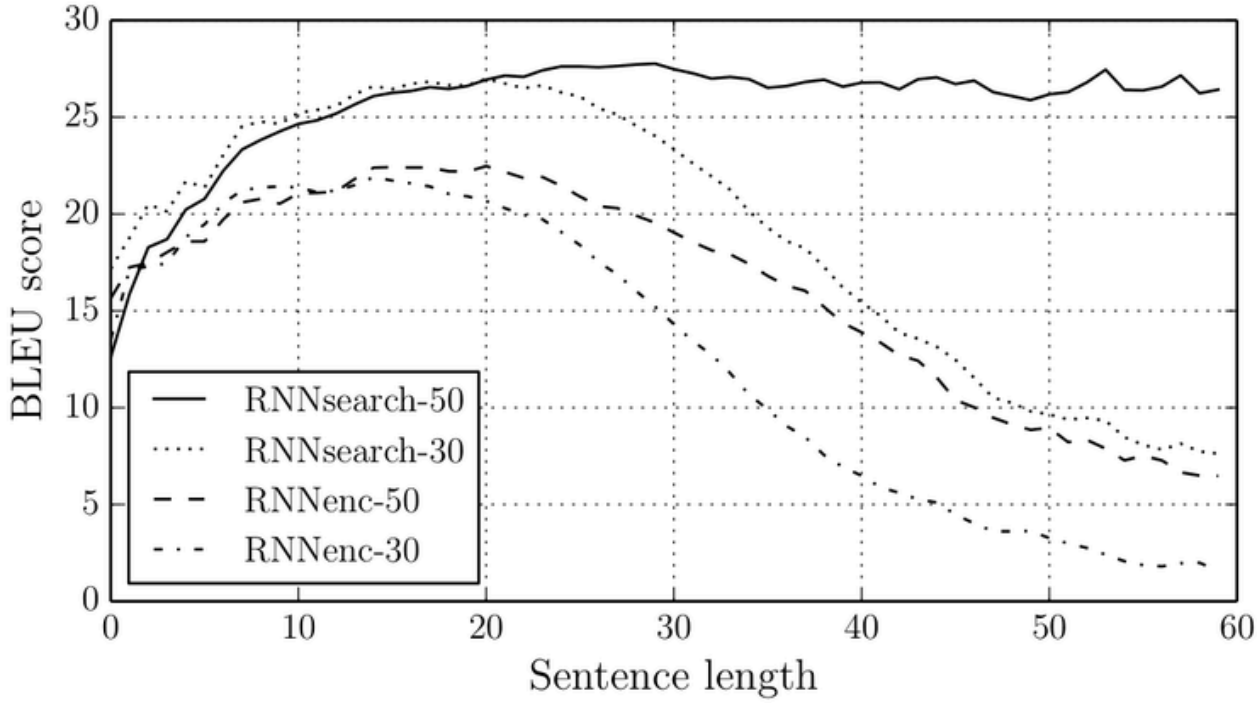
Bahdanau, Cho and Bengio (2014), Figure 3a. Rows: the French words written. Columns: the English words. Brighter: more weight.

Think first. Find “European Economic Area”. In which order does the model look at these three words? Show the answer

Result

Encoder-decoder	26.71
With attention	34.16

BLEU score on test sentences with no unknown words. Higher is better. Source: Bahdanau, Cho and Bengio (2014), Table 1.



Bahdanau, Cho and Bengio (2014), Figure 2. Score by sentence length. **RNNenc**: the plain encoder-decoder, which falls on long sentences. **RNNsearch**: with attention, which holds up.

STORY · 2014

From RNNsearch to “attention”

Bahdanau named the model RNNsearch. The far better name, **attention**, was added by Bengio in a final revision.

KNOWLEDGE CHECK

With attention, what does the decoder receive at each output step that a plain encoder-decoder never gives it?

A The final encoder hidden state, passed in once at the start

B The word it produced at the previous step, as its next input

C A probability for every vocabulary word, computed with softmax

D A weighted mix of all encoder states, recomputed for that step

PART 2 · 2016

The bold move

3 The encoder is still an RNN

Problem 3 of 9

Attention fixed how the decoder sees the input. The encoder still reads one word at a time.

PROBLEM Distant words are linked only through a long chain of steps, and each step must wait for the one before.

Word 1 must reach word 30

Reset | Previous | Next | Play



Word 1 has to reach word 30. Press **Next**.

KEY IDEA **Long chains fade, and waiting in line leaves the GPU idle.**

STORY · GOOGLE · 2016

The heretic



Jakob Uszkoreit
Google Research
He believed attention alone, with no recurrence at all, could beat recurrent networks. It meant throwing out every existing architecture. Even his father, a well-known computational linguist, disagreed with him over dinner.



Illia Polosukhin
Google Research · the lunch
He complained that answers on Google search had to come back within milliseconds, and RNNs were too slow. Uszkoreit's answer: why not use self-attention?

Fix · Every word attends to every word

FIX Remove recurrence. In **self-attention**, every word looks at every other word directly, all at the same time.

Every pair, all at once

Reset | Previous | Next | Play

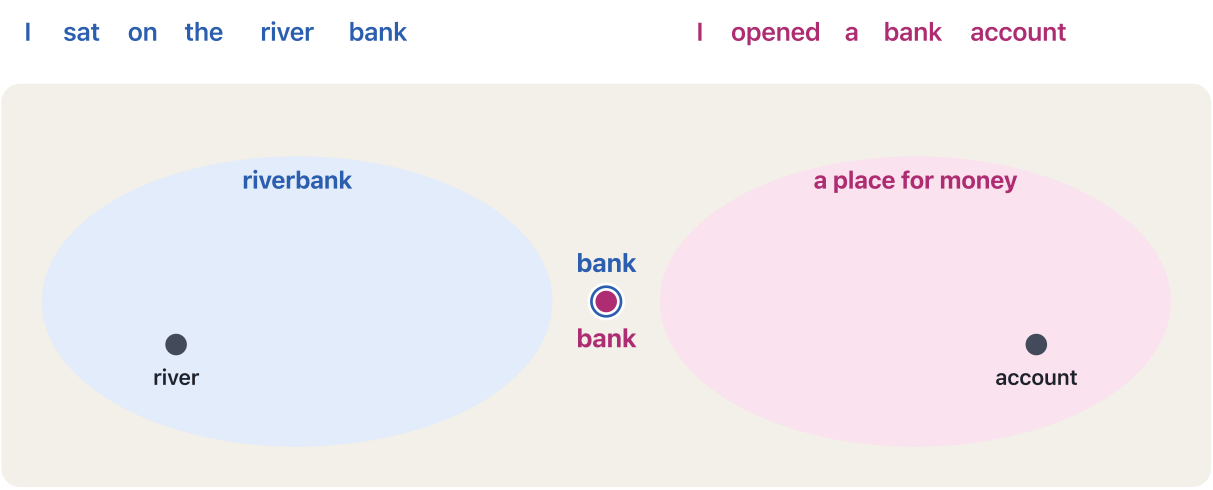


Uszkoreit's idea: drop recurrence. Each word starts with its own pattern. Press **Next**.

What self-attention produces

Each word takes a weighted mix of the words around it. So the same word can end up with different meanings in different sentences.

The same word, two meanings Reset Previous Next Play



Both sentences contain "bank". It starts from the same embedding, at the same point.

> KEY IDEA **The same word starts with the same embedding. After self-attention, it carries its meaning in context.**

What did we just lose?

Recurrence did four jobs for us without being asked. Self-attention threw them away.

Think first. Name the four jobs. Show the answer

PROBLEM 5 · WON BACK ✓

Word order

Recurrence read the words in order, so it always knew where each word sat.

PROBLEM 7 · WON BACK ✓

Processing each word

Every step passed the word through a nonlinear transformation.

PROBLEM 8 · WON BACK ✓

Depth that trains

We will stack many blocks, and the gradient has to survive them.

PROBLEM 9 · WON BACK ✓

No peeking

When writing, it could only see the words it had already written.

First, two questions about attention itself. How does a word decide what is relevant (Problem 4)? Can one attention capture several relationships at once (Problem 6)?

PART 3 · 2017

Winning back what we lost

STORY · GOOGLE · 2017

The team forms



Ashish Vaswani
Google Brain
Born in India, trained in machine translation at USC. With Illia Polosukhin, he designed and implemented the first Transformer models.



Niki Parmar
Google Research
Worked at a software company in India before her master's in the US. She designed, tuned and evaluated countless model variants.



Aidan Gomez
Intern, University of Toronto
An undergraduate who cold-emailed Google researchers, and learned months later that the internship was meant for PhD students.


Ashish Vaswani


Noam Shazeer


Niki Parmar


Jakob Uszkoreit


Llion Jones


Aidan Gomez


Łukasz Kaiser


Illia Polosukhin

The eight authors of the paper. The listed author order is random.

---

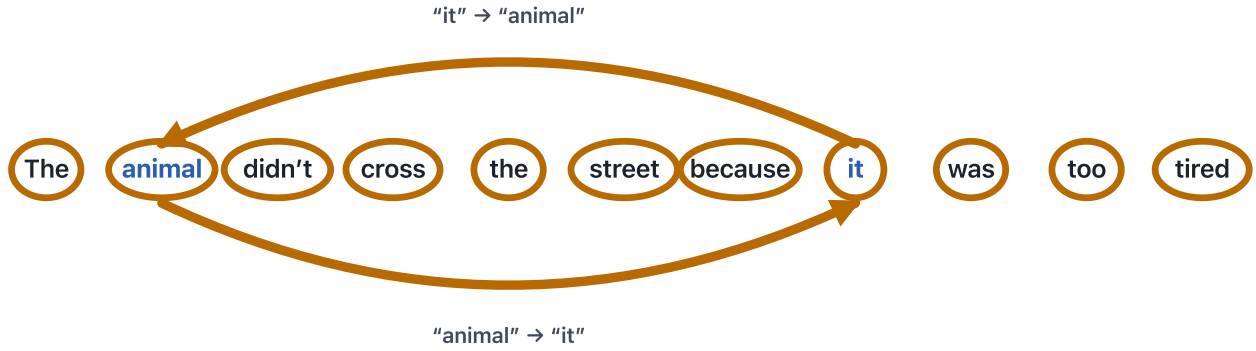
4 Relevance runs one way

Problem 4 of 9

Self-attention needs a score for every pair of words. The simplest score compares their two embeddings.

PROBLEM "it" needs "animal", but "animal" has no need for "it". Comparing embeddings gives both directions the same score, and every word matches itself best.

"it" and "animal" | Compare embeddings | Query, key, value



Comparing embeddings: “it” → “animal” scores exactly the same as “animal” → “it”, and every word matches itself best (thick rings). Now switch to **Query, key, value**.

Fix · Three roles for every word

FIX Each word produces three vectors, each from its own learned transformation.

Query

What this word is looking for.

Key

What this word offers, so other words can find it.

Value

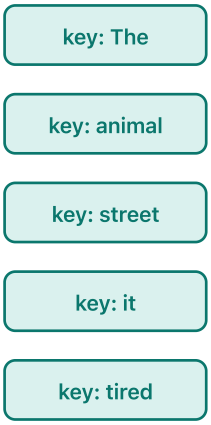
The information this word passes on once it is chosen.

The query of “it” is compared with the key of “animal”. The query of “animal” is compared with the key of “it”. These are different vectors, so the two directions get different scores.

The full recipe for one word

The recipe for “it”

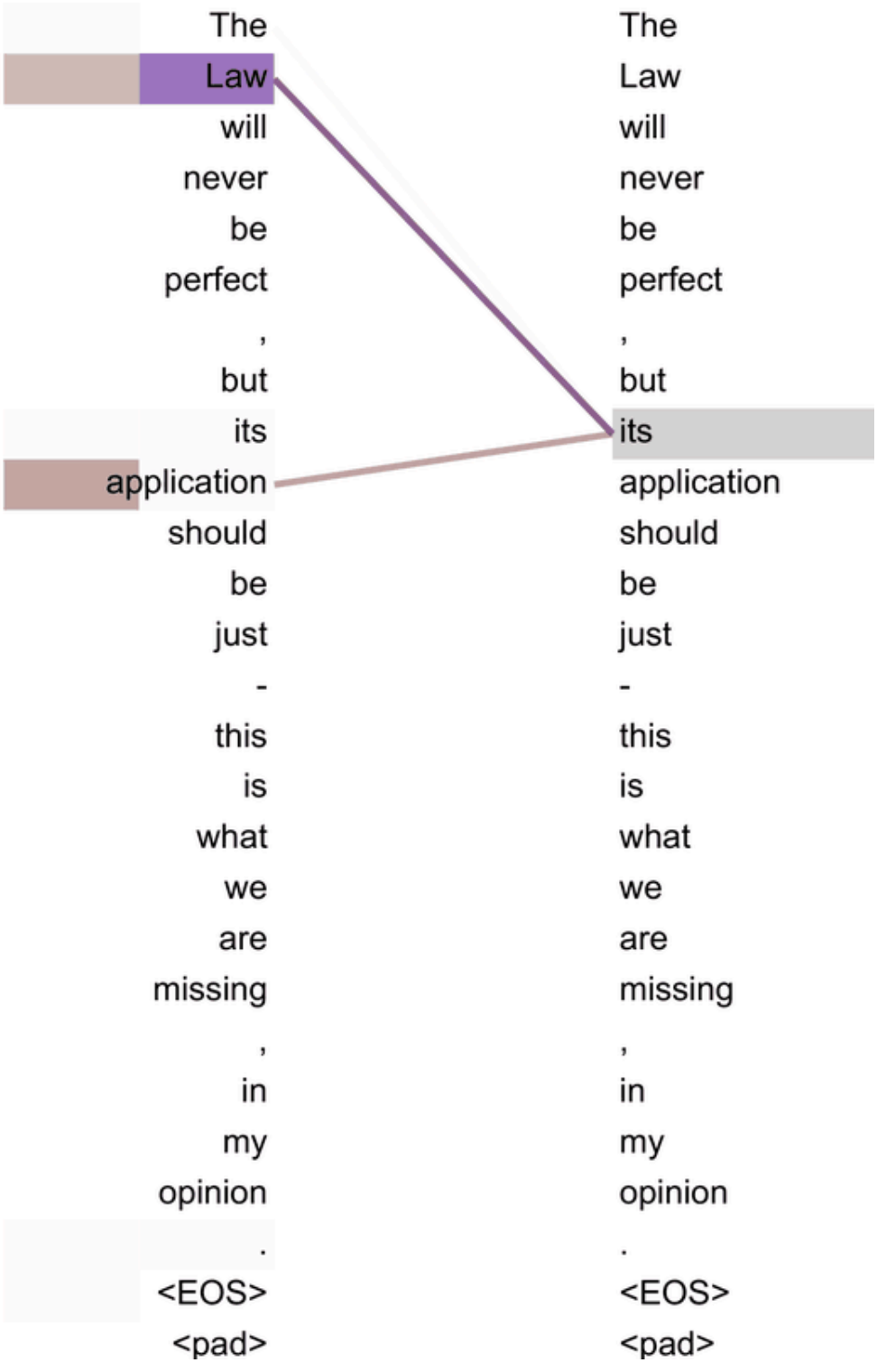
Reset | Previous | Next | Play



Every word runs this recipe at the same time. Follow “it”. Press **Next**.

> **KEY IDEA** **Every word runs this recipe at the same time. The new “it” is mostly “animal”.**

The trained Transformer does exactly this. In the paper’s model, “its” looks at the noun it refers to.



Vaswani et al. (2017), Figure 4 (bottom). One attention head of the trained model, reading “The Law will never be perfect, but its application should be just”. From “its”, the attention goes to “Law” and “application”.

KNOWLEDGE CHECK

In “The animal didn’t cross the street because it was too tired”, why can “it” attend strongly to “animal” while “animal” attends weakly to “it”?

A Multi-head attention lets separate heads follow different relationships in one sentence

B Positional encoding gives the two words different patterns for their different places

C Each word’s value carries the information it passes on once another word chooses it

D The query of one word and the key of another come from different learned transformations

5 Word order is gone

Problem 5 of 9

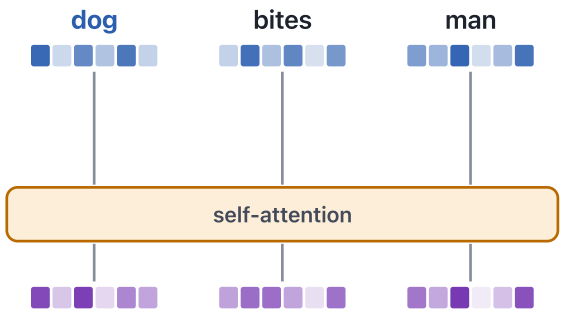
Wins back: word order

Recurrence always knew where each word sat. Self-attention does not.

PROBLEM Self-attention never looks at positions. Shuffle the words, and each word’s output stays exactly the same.

“dog bites man” and “man bites dog”

Reset Previous Next Play



Self-attention reads “dog bites man”. Each word becomes a pattern of numbers, shown as shades. Watch the output for “dog”.

FIX Add a **position vector** to each word’s embedding before attention. The paper builds these vectors from sine and cosine waves, so every position gets its own pattern.

> **KEY IDEA** **Now “dog” at position 1 and “dog” at position 3 start out different, and attention can use that difference.**

KNOWLEDGE CHECK

Without positional encoding, why does self-attention give “dog” the same output in “dog bites man” and “man bites dog”?

A Each word attends to every word in the sentence, including itself

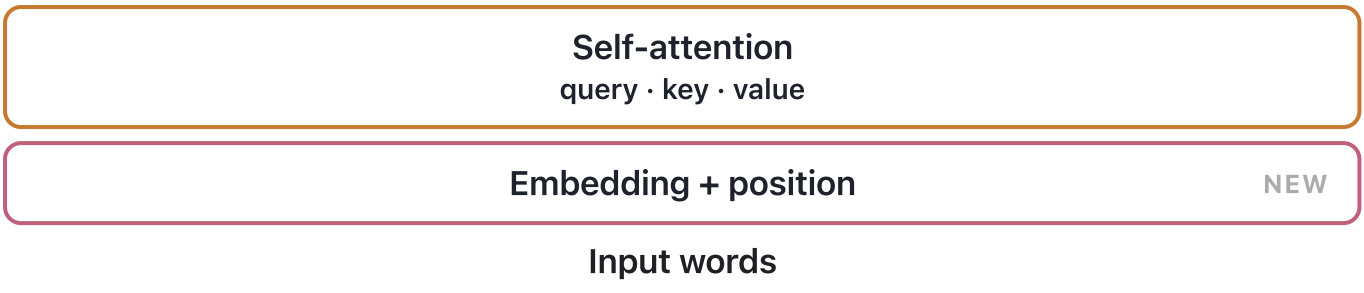
B All of the words in the sentence are processed at the same time

C Each word’s scores depend only on word content, never on position

D The attention weights for each word are normalised to sum to one

The block so far

Output vectors



Self-attention
Problem 3

Query, key and value
Problem 4

Position added to each word
Problem 5

STORY · GOOGLE · 2017

Good, but not better

The early Transformer models worked. They matched the best LSTM systems but, in Uszkoreit's words, they were not better. The team needed someone to turn a promising idea into a system that wins.



Noam Shazeer
Google, since 2000
Walking past a colleague's desk, he overheard Vaswani and Parmar discussing self-attention. He joined, rewrote the code himself, and came back saying it worked. The paper credits him with scaled dot-product attention, multi-head attention and the position representation.

---

6 One average cannot hold every relationship

Problem 6 of 9

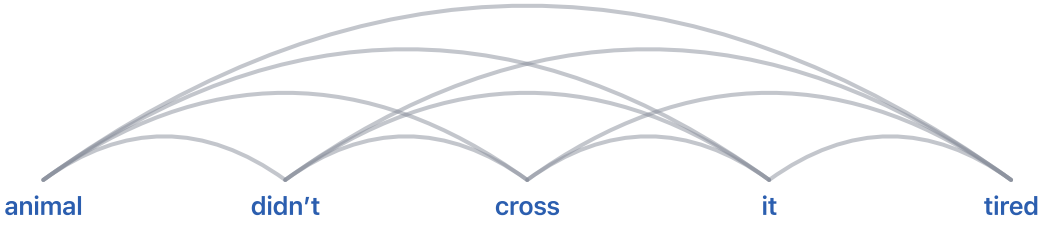
A word has several relationships at once. “it” refers to “animal”, “it” is described by “tired”, and “didn’t” belongs with “cross”.

PROBLEM One attention computes one weighted average, so every relationship blends into it.

Three relationships in one sentence

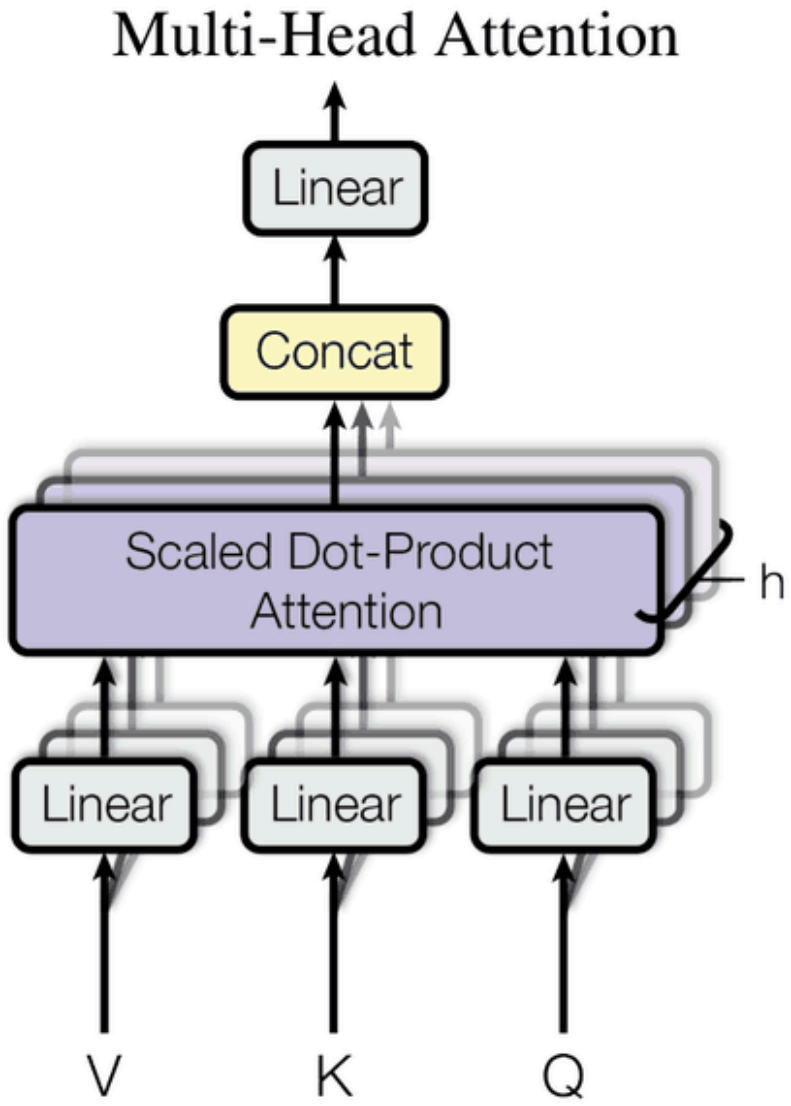
One head | Three heads

One set of weights: every relationship blends together



One head computes one weighted average. The three relationships blur into a single grey mix. Now switch to **Three heads**.

FIX Run several attention **heads** side by side, each with its own query, key and value transformations. Then join their outputs. The paper uses 8 heads.



Vaswani et al. (2017), Figure 2 (right). Multi-head attention: **h** heads run in parallel, then their outputs are joined (Concat).

KEY IDEA **Each head learns its own relationship.**

7 Attention only mixes

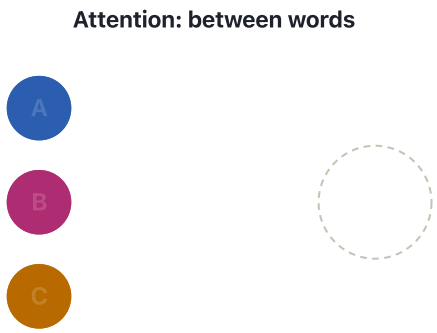
Problem 7 of 9

Wins back: processing

In the RNN, every step passed the word through a nonlinear transformation.

PROBLEM The output of attention is a weighted average of value vectors. Information moves between words, but no word is transformed.

Communication, then processing Reset Previous Next Play



Attention passes information between words through their values A, B and C.

FIX After attention, pass each word through a small **feed-forward network**. The same network is applied to every position separately.

> **KEY IDEA** **Attention is communication between words. The feed-forward network is processing within each word.**

---

8 Deep stacks are hard to train

Problem 8 of 9

Wins back: depth

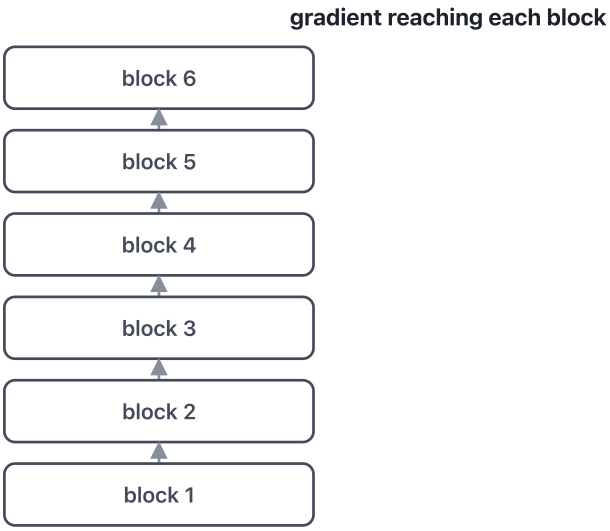
One block is not enough. The Transformer stacks six.

PROBLEM In training, the gradient must flow back through every block, and it shrinks on the way.

The gradient flowing back

Without skip connections | With skip connections Reset Previous

Next Play



Six blocks are stacked. In training, the gradient flows back from the top. Press **Next**.

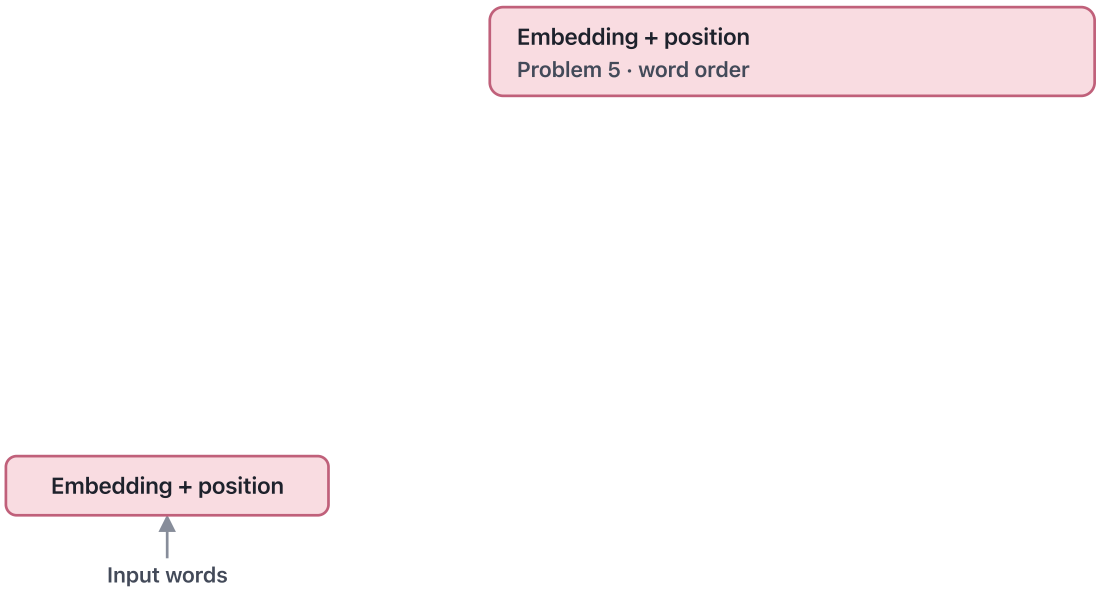
FIX Wrap every sub-layer in **Add & Norm**. The skip connection adds the input back to the output, so the gradient has a direct path. Layer norm keeps the values in a steady range.

You have seen this idea before. The LSTM's long-term memory also adds instead of rewriting, and that is why its learning signal survived long gaps.

> **KEY IDEA** **Add, do not replace. The gradient reaches every block.**

The encoder block, complete

Building the encoder Reset Previous Next Play



Each word becomes an embedding, plus its position (Problem 5).

PART 4 · 2017

The decoder

9 The decoder can see the answer

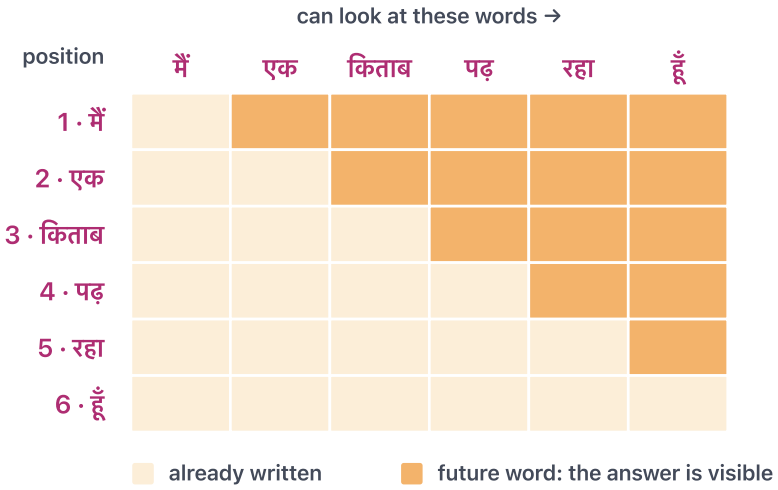
Problem 9 of 9

Wins back: no peeking

The decoder writes Hindi one word at a time. In training, it receives the whole correct Hindi sentence at once, so that it can learn every position in parallel.

PROBLEM Self-attention lets every position see every word, including the future words it is supposed to predict.

What each position can see No mask | Mask



Without a mask, every position can see the words after it. In training, the decoder can simply copy the next word. Now switch to **Mask**.

FIX **Mask** the future. Before softmax, every future cell is blocked, so its weight becomes 0.

KEY IDEA **Each position sees only the words already written.**

STORY · GOOGLE · 2017

A bug they actually hit



Aidan Gomez
The intern
He recalled the last weeks as rapid trial and error, including a model that behaved strangely because the masking had not been done properly.

KNOWLEDGE CHECK

During training, what does the mask in the decoder's self-attention prevent?

A Every relationship in a sentence from blending into one average

B Each Hindi position from attending to the words that come after it

C Each word from losing its position once all words are processed together

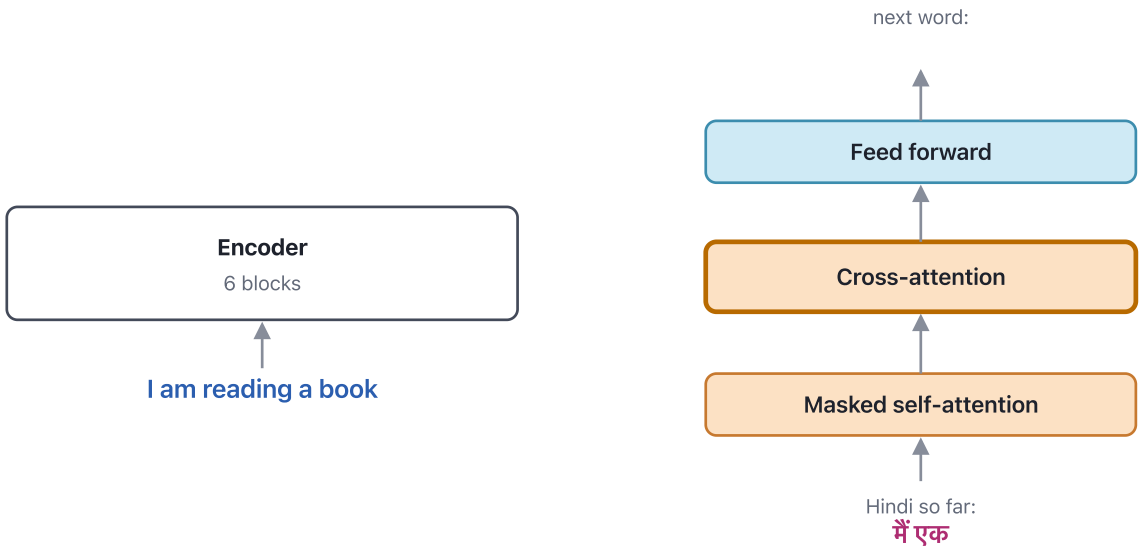
D The gradient from fading as it passes back through six stacked blocks

Cross-attention: 2014 returns

The decoder has one more attention layer. It connects the decoder to the encoder, just like Bahdanau's attention in Problem 2.

Cross-attention Reset Previous Next Play

I am reading a book → मैं एक किताब पढ़ रहा हूँ



The decoder has three parts. The middle one connects it to the encoder.

> KEY IDEA **The idea that started the story sits inside the final architecture.**

STORY · MAY 2017

Two nights before the deadline



Llion Jones
Google Research

The NeurIPS deadline of 19 May was close and the paper still had no title. He noted the team had thrown out LSTMs for one

technique, and borrowed from the Beatles: "Attention Is All You Need."

5 min before submission, the last English-French numbers arrived.

2 min to spare when they sent the paper off.

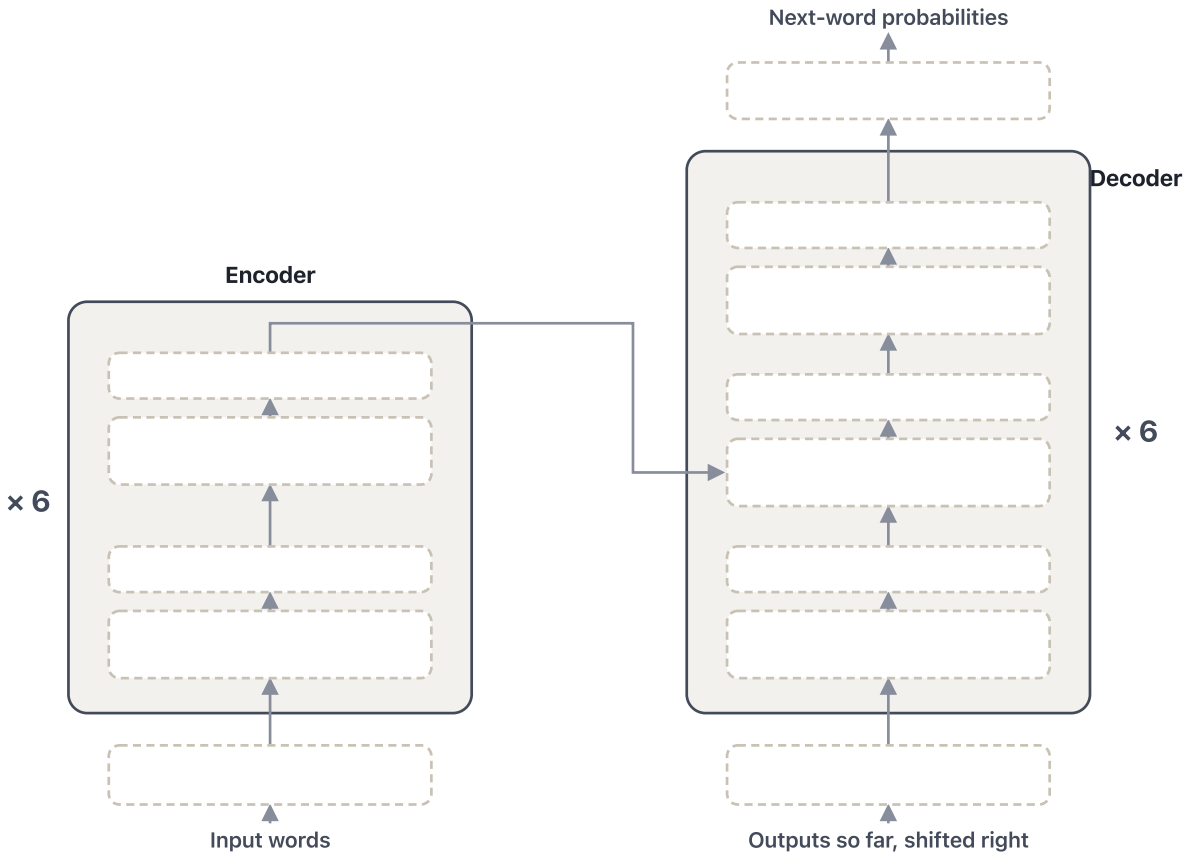
Label every box

Every piece is now on the table. Choose a label, then click the box it belongs to.

The complete Transformer

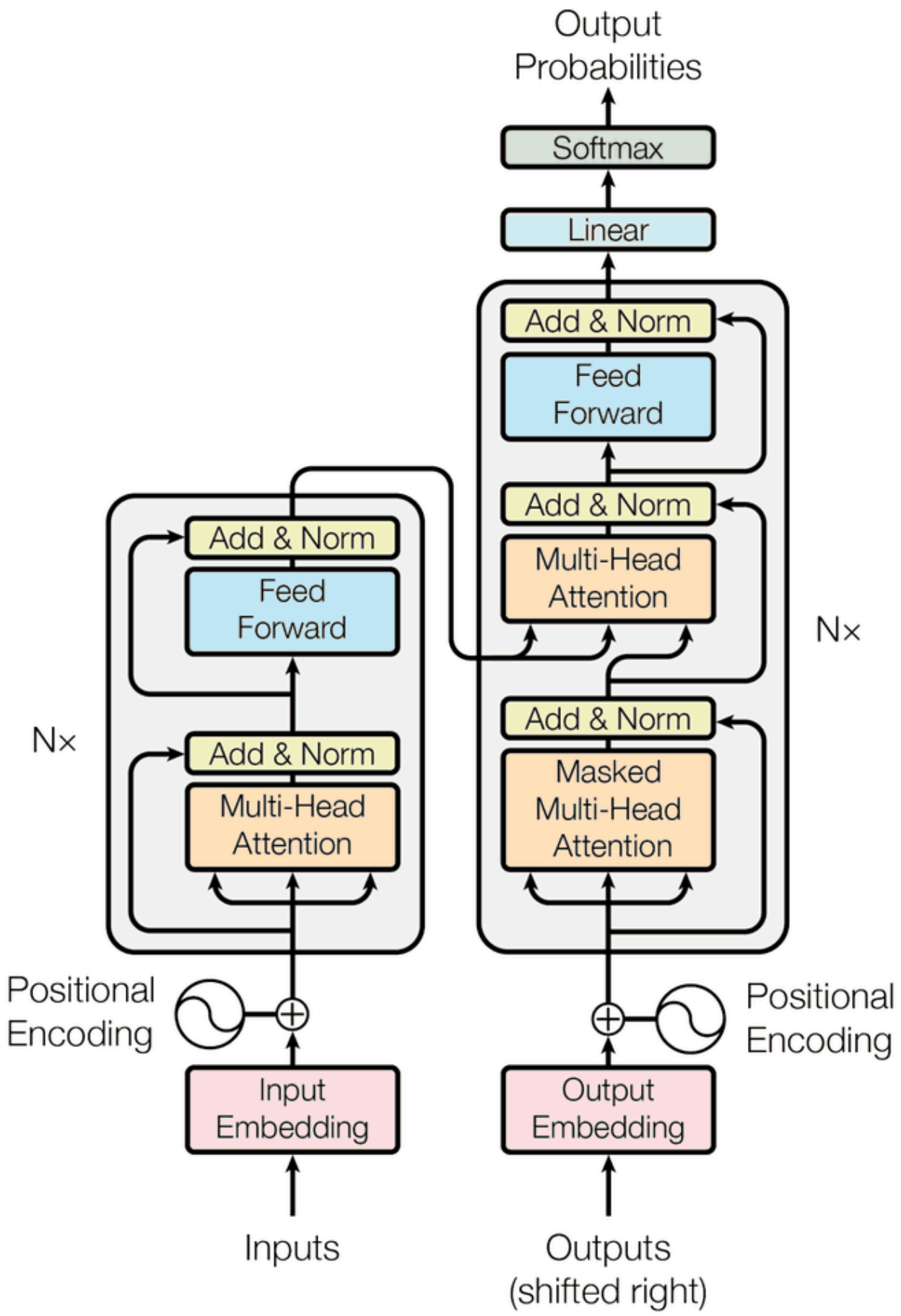
Reset | Show all

Embedding + position | Multi-head attention | Masked multi-head attention | Cross-attention | Add & Norm | Feed forward | Linear + softmax



Choose a label, then click its box. **0 of 13** placed.

Compare your diagram with the original Figure 1 of the paper.



Vaswani et al. (2017), Figure 1. The Transformer. Left: the encoder. Right: the decoder. N = 6.

STORY · DECEMBER 2017

Drawn by the intern



Aidan Gomez
The undergraduate intern
The diagrams and visualisations in the paper were produced by Gomez, someone your age.

The reviews: one positive, one extremely positive, and one that said it was OK.

The slot: an evening poster session at NeurIPS in Long Beach, where Sepp Hochreiter came over to praise it.

It beat every earlier model

ByteNet	23.75
GNMT + RL	24.6
ConvS2S	25.16
MoE	26.03
GNMT + RL ensemble	26.3
ConvS2S ensemble	26.36
Transformer (base)	27.3
Transformer (big)	28.4

BLEU, WMT 2014 English to German (newstest2014). Bars start at 20 BLEU. Source: Vaswani et al. (2017), Table 2.

Training cost
Transformer (base): 12 hours on 8 GPUs, about 23 times less compute than the ConvS2S ensemble it beat.

Single model
The big model beat every earlier result, including ensembles of several models.

Epilogue: who built on it

2014

Attention

Bahdanau

2017

The Transformer

Vaswani et al.

2018

GPT and BERT

OpenAI, Google

2022

ChatGPT

OpenAI

GPT

Encoder | **Decoder**

Keeps only the decoder. It writes text one word at a time, with masked self-attention.

BERT

Encoder | Decoder

Keeps only the encoder. It reads the whole sentence at once to understand it.

The original Transformer

Encoder | **Decoder**

Uses both halves, for translation: the encoder reads and the decoder writes.

STORY · 2018 ONWARDS

The rival returns



Ilya Sutskever
OpenAI
The rival from the 2014 race. At OpenAI, he suggested that Alec Radford work on the Transformer.

AR

Alec Radford
OpenAI
The result was the first GPT: a stack of Transformer decoders trained to predict the next word.

Everyone left. Google put Transformers into Translate and BERT in 2018, but all eight authors eventually left the company.

Summary: every component, every problem

#	COMPONENT	PROBLEM IT SOLVES
1	**Encoder-decoder**	Input and output lengths differ, and the output needs the whole input.
2	**Attention (cross-attention)**	One fixed vector cannot hold a long sentence.
3	**Self-attention**	Recurrence is slow and links distant words through long chains.
4	**Query, key, value**	Relevance must be learned and one-directional.
5	**Positional encoding**	Self-attention cannot see word order.
6	**Multi-head attention**	One weighted average cannot capture several relationships.
7	**Feed-forward network**	Attention only mixes; each word also needs processing.
8	**Residual + layer norm**	Deep stacks are hard to train.
9	**Masking**	The decoder could see the answer during training.

Exit ticket

Three questions. One click each.

EXIT TICKET · QUESTION 1

Which component of the Transformer exists because attention only mixes information between words?

A Residual connections, giving gradients a direct path back through the stack

B The feed-forward network, which processes each position's vector on its own

C Masking, which stops each decoder position from attending to later positions

D Layer normalisation, which keeps each vector's values in a stable range

EXIT TICKET · QUESTION 2

Which component lets the decoder look at the English sentence while it writes Hindi?

A Masked self-attention, which lets each Hindi word look at earlier Hindi words

B The feed-forward network, which processes each word's vector on its own

C Cross-attention, which lets each Hindi word look at the encoder's output

D Positional encoding, which tells each word where it sits in the sentence

EXIT TICKET · QUESTION 3

Why can the Transformer encoder process all the words of a sentence at the same time?

A Self-attention links every pair of words directly, with no step waiting for another

B Positional encoding gives every word a pattern that marks its place in the sentence

C Residual connections give the gradient a direct path back through the stacked blocks

D Multi-head attention lets several separate heads track different relationships between the words

NEXT LECTURE **The maths inside each box: how scores, softmax, queries, keys and values are computed.**

The Transformer · interactive worksheet.

Figures reproduced with attribution for teaching: A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, “Attention Is All You Need”, NeurIPS 2017 (Google grants permission to reproduce its tables and figures for scholarly use with attribution). D. Bahdanau, K. Cho and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate”, ICLR 2015.

Photos: Sepp Hochreiter: C.Stadler/Bwag / CC BY-SA 4.0, via Wikimedia Commons; Ilya Sutskever: The Royal Society (Fellow profile); Yoshua Bengio: Xuthoria / CC BY-SA 4.0, via Wikimedia Commons; Dzmitry Bahdanau: Mila (member directory); Jakob Uszkoreit: Handelsblatt Tech Europe (speaker page); Illia Polosukhin: CfC St. Moritz (speaker profile); Ashish Vaswani: Computerworld (Essential AI photo of Vaswani and Parmar); Niki Parmar: Computerworld (Essential AI photo of Vaswani and Parmar); Aidan Gomez: Gabriel Hutchinson / CC BY-SA 4.0, via Wikimedia Commons; Noam Shazeer: Fortune (Winni Wintermeyer for The Washington Post via Getty Images); Llion Jones: Syced / CC0, via Wikimedia Commons; Łukasz Kaiser: Ingeniøren Digital Tech Summit (speaker page).